

QUESTIONS

10 / 10 completed

Fibonacci series ($N \leq 50$)FIBONACCI SERIES ($N \leq 50$)

Develop a C program to print the first N terms of the Fibonacci series ($N \leq 50$) without using recursion. Use two variables to generate the series iteratively and store the series in an array. The program should also calculate and display the sum of the series and identify the largest term. Input validation is mandatory for N ($1 \leq N \leq 50$).

Sample Input/Output: text

Enter number of terms (1-50): 10

Fibonacci Series: 0 1 1 2 3 5 8 13 21 34

Sum of series: 88

Largest term: 34

Enter number of terms (1-50): 60

Invalid input! Please enter terms between 1 and 50.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5     long long a = 0, b = 1, c, sum = 0, largest = 0;
6
7     scanf("%d", &n);
8
9     if (n < 1 || n > 50) {
10         printf("Invalid input! Please enter terms between 1 and 50.");
11         return 0;
12     }
13
14     printf("Fibonacci Series: ");
15
16     for (i = 1; i <= n; i++) {
17         if (i == 1)
18             c = 0;
19         else if (i == 2)
20             c = 1;
21         else {
```

OUTPUT

```
Fibonacci Series: 0 1 1 2 3 5 8 13 21 34
Sum of series: 88
Largest term: 34
```

INPUT

QUESTIONS

10 / 10 completed

Unsorted Array

UNSORTED ARRAY

Write a program to find the missing element for the given unsorted array of sequence numbers.

Sample Input: a=[10,9,7,4,3,1,2]

Sample output: 5,6,8

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5     long long a = 0, b = 1, c, sum = 0, largest = 0;
6
7     scanf("%d", &n);
8
9     if (n < 1 || n > 50) {
10         printf("Invalid input! Please enter terms between 1 and 50.");
11         return 0;
12     }
13
14     printf("Fibonacci Series: ");
15
16     for (i = 1; i <= n; i++) {
17         if (i == 1)
18             c = 0;
19         else if (i == 2)
20             c = 1;
21         else {
```

OUTPUT

```
Fibonacci Series: 0 1 1 2 3 5 8 13 21 34
Sum of series: 88
Largest term: 34
```

INPUT

10

QUESTIONS

10 / 10 completed

Sparse Matrices

SPARSE MATRICES

Represent two sparse matrices (mostly zeros) using arrays of triples [row, col, value]. Read dimensions and non-zero terms for two matrices. Add them, store result in sparse form (sorted by row then col), and display dense matrix form. Max 10x10; ignore zeros.

Sample Input (Matrix 1):

```
3 3 3
0 1 5
1 0 8
2 2 2
```

Sample Input (Matrix 2):

```
3 3 2
0 1 3
2 2 7
```

Sample Output (Sum Dense):

```
0 8 0
8 0 0
0 0 9
```

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int r1, c1, t1, r2, c2, t2;
5     int a[10][10] = {0}, b[10][10] = {0}, sum[10][10] = {0};
6     int i, row, col, val;
7
8     // Matrix 1
9     scanf("%d %d %d", &r1, &c1, &t1);
10    for(i = 0; i < t1; i++) {
11        scanf("%d %d %d", &row, &col, &val);
12        a[row][col] = val;
13    }
14
15    // Matrix 2
16    scanf("%d %d %d", &r2, &c2, &t2);
17    for(i = 0; i < t2; i++) {
18        scanf("%d %d %d", &row, &col, &val);
19        b[row][col] = val;
20    }
21}
```

OUTPUT

```
0 8 0
8 0 0
0 0 9
```

INPUT

3 3 3

QUESTIONS

10 / 10 completed

Fibonacci Series

FIBONACCI SERIES

Write a C program to print the first n terms of the Fibonacci series ($n \leq 20$) without using recursion. Start with 0 and 1.

Sample Input/Output:

Enter number of terms: 8

Fibonacci series: 0 1 1 2 3 5 8 13

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, a = 0, b = 1, c, i;
5
6     scanf("%d", &n);
7
8     for (i = 1; i <= n; i++) {
9         if (i == 1)
10            printf("%d ", a);
11        else if (i == 2)
12            printf("%d ", b);
13        else {
14            c = a + b;
15            printf("%d ", c);
16            a = b;
17            b = c;
18        }
19    }
20
21    return 0;
}
```

OUTPUT

0 1 1 2 3 5 8 13

INPUT

8



QUESTIONS

10 / 10 completed

Array



ARRAY

Write a C program to find the 5th element in the given array/

Sample Input: a=[10,20,30,40,50,60]

Sample output: 50

PROGRAM EDITOR

C



Run

Save

```
1 #include<stdio.h>
2 int main()
3 {
4     int a[6]
5     for(int i=0;i<6;i++)
6         scanf("%d",&a[i]);
7     printf("%d",a[4]);
8     return 0;
9 }
```

OUTPUT

Compilation Error:

```
C:\Windows\TEMP\sandbox_192525253RD_6a55c5635cac79.65717984\main.c: In function 'main':
C:\Windows\TEMP\sandbox_192525253RD_6a55c5635cac79.65717984\main.c:5:5: error: expected
'=', ',', '(', '}', 'asm' or '__attribute__' before 'for'
```

```
5 |     for(int i=0;i<6;i++)
  |     ^~~~
```

```
C:\Windows\TEMP\sandbox_192525253RD_6a55c5635cac79.65717984\main.c:6:19: error: 'a'
undeclared (first use in this function)
```

```
6 |         scanf("%d",&a[i]);
  |                   ^
```

INPUT

[10 20 30 40 50 60]

QUESTIONS

10 / 10 completed

Integer Array

INTEGER ARRAY

"Write a C program to demonstrate pointer arithmetic on an integer array. Display element values using both array indexing and pointer notation. Show address difference between consecutive elements.

Sample Input:

Enter number of elements: 4

Enter elements:

10 20 30 40

Sample Output:

Using Array Indexing:

arr[0] = 10

arr[1] = 20

arr[2] = 30

arr[3] = 40

Using Pointer Notation:

*(ptr+0) = 10

*(ptr+1) = 20

*(ptr+2) = 30

*(ptr+3) = 40

Address Difference:

Address difference between arr[0] and arr[1] = 4 bytes"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5     int arr[100];
6     int *ptr;
7
8     // Input
9     scanf("%d", &n);
10
11     for(i = 0; i < n; i++) {
12         scanf("%d", &arr[i]);
13     }
14
15     ptr = arr;
16
17     // Using Array Indexing
18     printf("Using Array Indexing:\n");
19     for(i = 0; i < n; i++) {
20         printf("arr[%d] = %d\n", i, arr[i]);
21     }
```

OUTPUT

```
Using Array Indexing:
arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40

Using Pointer Notation:
*(ptr+0) = 10
*(ptr+1) = 20
*(ptr+2) = 30
```

INPUT

QUESTIONS

10 / 10 completed

Factorial

FACTORIAL

*Write a C program to find the factorial of a given number using recursion.

Sample Input: Enter a number- 6

Sample Output: Factorial of 6 is 720

*

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 int fact(int n)
3 {
4     if(n==0||n==1)
5         return 1;
6     return n * fact(n-1);
7 }
8 int main()
9 {
10     int n;
11     scanf("%d",&n);
12     printf("fctorial of %d is %d",n, fact(n));
13     return 0;
14 }
```

OUTPUT

Compilation Error:

C:\Windows\TEMP\sandbox_192525253RD_6a55c3c7285852.26229381\main.c: In function 'main':

C:\Windows\TEMP\sandbox_192525253RD_6a55c3c7285852.26229381\main.c:12:44: error:

expected ';' before ':' token

```
12 |     printf('fctorial of %d is %d',n, fact(n));
    |                                     ^
    |                                     ;
```

INPUT

QUESTIONS

10 / 10 completed

Array Rotation

ARRAY ROTATION

"Implement array rotation (left/right) for arrays of size 10 by K positions without using extra space. Use a reversal algorithm.

The program must validate K ($0 \leq K < N$), show

original → rotated → reversal steps visually, compute the sum difference, and support multiple rotations.

Sample Input/Output:

Enter 10 numbers: 1 2 3 4 5 6 7 8 9 10

Rotate Left (L)/Right (R): L

By positions: 3

Original: [1 2 3 4 5 6 7 8 9 10]

Step1: Reverse 0-2 → [3 2 1 4 5 6 7 8 9 10]

Step2: Reverse 3-9 → [3 2 1 10 9 8 7 6 5 4]

Step3: Reverse all → [4 5 6 7 8 9 10 1 2 3]

Sum difference: 0"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 void reverse(int arr[], int start, int end) {
4     while (start < end) {
5         int temp = arr[start];
6         arr[start] = arr[end];
7         arr[end] = temp;
8         start++;
9         end--;
10    }
11 }
12
13 void printArray(int arr[], int n) {
14     int i;
15     printf("[ ");
16     for (i = 0; i < n; i++)
17         printf("%d ", arr[i]);
18     printf("]\n");
19 }
20
21 int main() {
```

OUTPUT

```
Enter 10 numbers: Rotate Left (L) / Right (R): By positions: Original: [ 1 2 3 4 5 6 7
8 9 10 ]
Step1: Reverse 0-2 -> [ 3 2 1 4 5 6 7 8 9 10 ]
Step2: Reverse 3-9 -> [ 3 2 1 10 9 8 7 6 5 4 ]
Step3: Reverse all -> [ 4 5 6 7 8 9 10 1 2 3 ]
Sum difference: 0
```

INPUT

1 2 3 4 5 6 7 8 9 10

QUESTIONS

10 / 10 completed

Binary Search- Iterative

BINARY SEARCH- ITERATIVE

Read n daily temperatures into an array, sort ascending using any method. Use binary search (iterative) to find first day $\geq 30^{\circ}\text{C}$ and last day $\leq 25^{\circ}\text{C}$. Display temperatures, indices (0-based), or "Not found". Validate sorted; $n \leq 50$.

Sample Input: 6

22 28 31 24 29 32

Sample Output (after sort):

22 24 28 29 31 32

First ≥ 30 : index 4 (31)Last ≤ 25 : index 1 (24)

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, a[50], i, j, temp;
5     int low, high, mid;
6     int first = -1, last = -1;
7
8     scanf("%d", &n);
9
10    if (n < 1 || n > 50) {
11        printf("Invalid input");
12        return 0;
13    }
14
15    for (i = 0; i < n; i++)
16        scanf("%d", &a[i]);
17
18    // Sort array (Bubble Sort)
19    for (i = 0; i < n - 1; i++) {
20        for (j = 0; j < n - i - 1; j++) {
21            if (a[j] > a[j + 1]) {
```

OUTPUT

```
Sorted: 22 24 28 29 31 32
First >=30: index 4 (31)
Last <=25: index 1 (24)
```

INPUT

QUESTIONS

10 / 10 completed

Linear Search

LINEAR SEARCH

Create a C program to perform Linear Search on an array of 10 integers. The program should allow manual input or random generation of array elements. It must display the first position of the element (1-based indexing), total comparisons made, search status (Found/Not Found), and time taken in milliseconds.

Sample Input:

Enter 1 for manual input, 2 for random: 1

Enter 10 Integers:

45 23 67 12 89 34 56 78 90 11

Enter target value: 34

Sample Output:

Target 34 found at position 6

Comparisons made: 6

Status: Found

Time taken: 0.002 ms

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int main() {
6     int a[10], i, choice, key;
7     int pos = -1, comparisons = 0;
8     clock_t start, end;
9     double time_taken;
10
11     scanf("%d", &choice);
12
13     if (choice == 1) {
14         for (i = 0; i < 10; i++)
15             scanf("%d", &a[i]);
16     } else if (choice == 2) {
17         srand(time(NULL));
18         for (i = 0; i < 10; i++)
19             a[i] = rand() % 100;
20     } else {
21         return 0;
22     }
```

OUTPUT

```
Target 34 found at position 6
Comparisons made: 6
Status: Found
Time taken: 0.000 ms
```

INPUT

1

QUESTIONS

10 / 10 completed

Binary Search

BINARY SEARCH

"Write a C program implementing binary search on a sorted array of 15 integers. The program must: (1) accept an unsorted array and sort it first, (2) perform binary search, (3) display search iterations with mid-point calculations, (4) show final result with comparison count, and (5) verify the array remains sorted post-search.

Sample Input/Output:

Enter 15 numbers: 50 30 10 90 40 70 20 80 60 5 45 35 25 55 15

Sorted array: 5 10 15 20 25 30 35 40 45 50 55 60 70 80 90

Enter target: 45

Binary Search Iterations:

low=0, high=14, mid=7, arr[7]=40

low=8, high=14, mid=11, arr[11]=55

low=8, high=10, mid=9, arr[9]=50

low=8, high=8, mid=8, arr[8]=45

Found at index 8 (1-based: 9)

Comparisons: 4

Array remains sorted: Yes"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int main() {
6     int a[10], i, choice, key;
7     int pos = -1, comparisons = 0;
8     clock_t start, end;
9     double time_taken;
10
11     scanf("%d", &choice);
12
13     if (choice == 1) {
14         for (i = 0; i < 10; i++)
15             scanf("%d", &a[i]);
16     } else if (choice == 2) {
17         srand(time(NULL));
18         for (i = 0; i < 10; i++)
19             a[i] = rand() % 100;
20     } else {
21         return 0;
22     }
```

OUTPUT

```
Target 34 found at position 6
Comparisons made: 6
Status: Found
Time taken: 0.000 ms
```

INPUT

1

QUESTIONS

10 / 10 completed

Linear Search on Array

LINEAR SEARCH ON ARRAY

"Develop a C program to perform Linear Search on an array of N integers. Accept array size dynamically

Sample Input/Output

Enter size: 5

Array: 10 20 30 40 50

Enter target: 40

Comparison Trace:

10 ≠ 40

20 ≠ 40

30 ≠ 40

40 = 40

Target Found at Position 4

Comparisons Made = 4

Test Cases

- Element at first Index
- Element at last Index
- Element absent"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int n, key, i, pos = -1, count = 0;
6
7     printf("Enter size: ");
8     scanf("%d", &n);
9
10    int *arr = (int *)malloc(n * sizeof(int));
11
12    printf("Array: ");
13    for(i = 0; i < n; i++)
14        scanf("%d", &arr[i]);
15
16    printf("Enter target: ");
17    scanf("%d", &key);
18
19    printf("Comparison Trace:\n");
20    for(i = 0; i < n; i++) {
21        count++;
```

OUTPUT

```
Enter size: Array: Enter target: Comparison Trace:
10 != 40
20 != 40
30 != 40
40 = 40
Target Found at Position 4
Comparisons Made = 4
```

INPUT

5

QUESTIONS

10 / 10 completed

Linear Search - Anagrams

LINEAR SEARCH - ANAGRAMS

"Read n strings, use linear search to find all anagrams of a target string (same chars, any order). Display positions (1-based) and matching strings. Case-insensitive; n<=50.

Sample Input: 5

1) listen silent

2) race care

3) hello world

4) abc cba

5) Xyz

Target: silent

Sample Output:

Anagrams found at positions: 1 2

Matches: listen silent, race care"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 void sortString(char s[]) {
6     int i, j;
7     char temp;
8     int n = strlen(s);
9     for(i = 0; i < n - 1; i++) {
10         for(j = i + 1; j < n; j++) {
11             if(s[i] > s[j]) {
12                 temp = s[i];
13                 s[i] = s[j];
14                 s[j] = temp;
15             }
16         }
17     }
18 }
19
20 int isAnagram(char str1[], char str2[]) {
21     char a[100], b[100];
```

OUTPUT

Security Error: Disallowed code pattern detected.

INPUT

5

QUESTIONS

10 / 10 completed

Linear Search -Sort

LINEAR SEARCH -SORT

Read n book page counts into an array. Sort ascending using any method. Use linear search to find the first book having ≥ 300 pages and the last book having ≤ 150 pages. Display page counts and indices (0-based), or "Not found". Validate sorted order; $n \leq 50$.

Sample Input: 7

120 450 275 310 140 500 200

Sample Output (after sort): 120 140 200 275 310 450 500

First ≥ 300 : index 4 (310)Last ≤ 150 : index 1 (140)

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, a[50], i, j, temp;
5     int first = -1, last = -1;
6
7     scanf("%d", &n);
8
9     if (n < 1 || n > 50) {
10         printf("Invalid input");
11         return 0;
12     }
13
14     for (i = 0; i < n; i++)
15         scanf("%d", &a[i]);
16
17     // Sort the array (Bubble Sort)
18     for (i = 0; i < n - 1; i++) {
19         for (j = 0; j < n - i - 1; j++) {
20             if (a[j] > a[j + 1]) {
21                 temp = a[j];
```

OUTPUT

```
Sorted: 120 140 200 275 310 450 500
First >=300: index 4 (310)
Last <=150: index 1 (140)
```

INPUT

7

QUESTIONS

10 / 10 completed

Linear Search -Prices

LINEAR SEARCH -PRICES

*Read n product prices into an array. Sort ascending using any method. Use linear search to find the first price $\geq$ ₹500 and the last price $\leq$ ₹200. Display prices and indices (0-based), or "Not found". Validate sorted order; $n \leq 50$.

Sample Input: 7

150 650 220 90 500 780 180

Sample Output (after sort): 90 150 180 220 500 650 780

First $\geq$ 500: index 4 (500)Last $\leq$ 200: index 2 (180)*

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, a[50];
6     int i, j, temp;
7     int first500 = -1, last200 = -1;
8
9     // Read number of products
10    scanf("%d", &n);
11
12    if (n <= 0 || n > 50)
13        return 0;
14
15    // Read prices
16    for (i = 0; i < n; i++)
17        scanf("%d", &a[i]);
18
19    // Sort in ascending order (Bubble Sort)
20    for (i = 0; i < n - 1; i++)
21    {
```

OUTPUT

```
90 150 180 220 500 650 780
First >=500: index 4 ('500')
Last <=200: index 2 ('180')
```

INPUT

7

QUESTIONS

10 / 10 completed

Linear Search -Temperature

LINEAR SEARCH -TEMPERATURE

"Read n daily temperatures into an array. Use Linear Search to find the last day whose temperature is less than 20°C. Display temperatures, index (0-based), and value. If not found, print "Not found". Validate $n \leq 50$.

Sample Input: 7

18 25 21 19 30 17 24

Sample Output: 18 25 21 19 30 17 24

Last <20: index 5 (17)"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, temp[50];
6     int i, lastIndex = -1;
7
8     // Read number of days
9     scanf("%d", &n);
10
11    // Validate n
12    if (n > 50 || n <= 0)
13        return 0;
14
15    // Read temperatures
16    for (i = 0; i < n; i++)
17    {
18        scanf("%d", &temp[i]);
19    }
20
21    // Display temperatures
```

OUTPUT

```
18 25 21 19 30 17 24
Last <20: index 5 (17)
```

INPUT

QUESTIONS

10 / 10 completed

Binary Search - Array

BINARY SEARCH - ARRAY

Write a C program to perform Binary Search on a sorted array of 10 integers. The program should accept the array elements and a target value from the user. It must display the position of the element and the total number of comparisons made.

Sample Input:

Enter 10 sorted integers:

5 12 18 23 34 45 56 67 78 90

Enter element to search: 45

Sample Output:

Element 45 found at position 6

Comparisons made: 3

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int arr[10], key;
5     int low = 0, high = 9, mid;
6     int comparisons = 0;
7     int found = 0;
8
9     // Input 10 sorted integers
10    for (int i = 0; i < 10; i++) {
11        scanf("%d", &arr[i]);
12    }
13
14    // Input element to search
15    scanf("%d", &key);
16
17    // Binary Search
18    while (low <= high) {
19        comparisons++;
20        mid = (low + high) / 2;
21    }
```

OUTPUT

```
Element 45 found at position 6
Comparisons made: 3
```

INPUT

5 12 18 23 34 45 56 67 78 90

QUESTIONS

10 / 10 completed

Linked List

LINKED LIST

"A library stores book identification numbers in a linked list. Develop a C program that searches for a given element and displays its position if found.

Test Case 1

Input

List: 10 → 20 → 30 → 40

Search Element: 30

Output

Element found at position 3"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 // Insert at end
13 void insertEnd(int value)
14 {
15     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->next = NULL;
18
19     if (head == NULL)
20     {
21         head = newNode;
```

OUTPUT

Element found at position 3

INPUT

10 20 30 40

QUESTIONS

10 / 10 completed

Single Linked List

SINGLE LINKED LIST

*A college maintains a list of student roll numbers in the order they are registered. New students may join at different stages of the registration process. Develop a C program using a singly linked list to store the roll numbers and perform insertion operations at the beginning, end, and specified position. Display the updated list after each insertion operation.

Test Case 1

Input

Initial List: Empty

Insert 10 at beginning

Insert 20 at end

Insert 15 at position 2

Output

10 -> 15 -> 20 -> NULL*

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 void insertBeginning(int value)
13 {
14     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
15     newNode->data = value;
16     newNode->next = head;
17     head = newNode;
18 }
19
20 void insertEnd(int value)
21 {
```

OUTPUT

10 -> 15 -> 20 -> NULL

INPUT

10 20 15

QUESTIONS

10 / 10 completed

Bin. Search-Iterative-Temp

BIN. SEARCH-ITERATIVE-TEMP

"Read n daily temperatures into an array, sort ascending using any method. Use binary search (iterative) to find first day $\geq 30^{\circ}\text{C}$ and last day $\leq 25^{\circ}\text{C}$. Display temperatures,

indices (0-based), or "Not found". Validate sorted; $n \leq 50$.

Sample Input: 6

22 28 31 24 29 32

Sample Output (after sort):

22 24 28 29 31 32

First ≥ 30 : Index 4 (31)

Last ≤ 25 : Index 1 (24)"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, a[50];
6     int i, j, temp;
7
8     scanf("%d", &n);
9
10    if (n <= 0 || n > 50)
11        return 0;
12
13    // Read temperatures
14    for (i = 0; i < n; i++)
15        scanf("%d", &a[i]);
16
17    // Sort in ascending order (Bubble Sort)
18    for (i = 0; i < n - 1; i++)
19    {
20        for (j = 0; j < n - i - 1; j++)
21        {
```

OUTPUT

Compilation Error:

C:\Windows\TEMP\sandbox_192525253RD_6a71a81610ce45.97095073\main.c: In function 'main':

C:\Windows\TEMP\sandbox_192525253RD_6a71a81610ce45.97095073\main.c:47:9: error:

expected expression at end of input

```
47 |         if (a[mid] >=
    |         ^~
```

C:\Windows\TEMP\sandbox_192525253RD_6a71a81610ce45.97095073\main.c:47:9: error:

expected declaration or statement at end of input

C:\Windows\TEMP\sandbox_192525253RD_6a71a81610ce45.97095073\main.c:47:9: error:

expected declaration or statement at end of input

INPUT

6

QUESTIONS

10 / 10 completed

Singly Linked List-Nodes



SINGLY LINKED LIST-NODES

"A hospital maintains patient records using a linked list. To generate reports, the system needs to determine the total number of records currently stored. Write a C program to count and display the total number of nodes in a singly linked list.

Test Case 1

Input

10 → 20 → 30 → 40 → 50

Output

Total number of nodes = 5"

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 // Insert at end
13 void insertEnd(int value)
14 {
15     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->next = NULL;
18
19     if (head == NULL)
20     {
21         head = newNode;
```

OUTPUT

Total number of nodes = 5

INPUT

10 20 30 40 50

QUESTIONS

10 / 10 completed

Singly Linked List-Reverse

SINGLY LINKED LIST-REVERSE

"In a logistics application, delivery locations are stored in a linked list. Sometimes the route needs to be traversed in reverse order. Write a C program to reverse a singly linked list and display the reversed list.

Test Case 1

Input

10 → 20 → 30 → 40

Output

40 → 30 → 20 → 10 → NULL

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 // Insert at end
13 void insertEnd(int value)
14 {
15     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->next = NULL;
18
19     if (head == NULL)
20     {
21         head = newNode;
```

OUTPUT

40 -> 30 -> 20 -> 10 -> NULL

INPUT

10 20 30 40

QUESTIONS

10 / 10 completed

Deletion Operations

DELETION OPERATIONS

"A company maintains employee records in a linked list. Employees may leave the organization, requiring their records to be removed. Write a C program to implement deletion operations from the beginning, end, and a specified position

Test Case 1

Input

List: 10 → 20 → 30 → 40

Delete node from beginning

Output

20 → 30 → 40 → NULL*

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 // Insert at end
13 void insertEnd(int value)
14 {
15     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->next = NULL;
18
19     if (head == NULL)
20     {
21         head = newNode;
```

OUTPUT

20 -> 30 -> 40 -> NULL

INPUT

10 20 30 40

QUESTIONS

10 / 10 completed

Circular Linked List

CIRCULAR LINKED LIST

"In a round-robin CPU scheduling system, processes are executed cyclically. Such applications can be represented using a circular linked list. Write a C program to create a circular linked list and display all nodes by traversing the list exactly once. Write a C program to create a circular linked list and traverse all nodes exactly once.

Test Case 1

Input

10 → 20 → 30 → 40

Output

10 → 20 → 30 → 40 → Back to Head"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 // Create Circular Linked List
13 void insertEnd(int value)
14 {
15     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
16     newNode->data = value;
17
18     if (head == NULL)
19     {
20         head = newNode;
21         newNode->next = head;
```

OUTPUT

10 -> 20 -> 30 -> 40 -> Back to Head

INPUT

QUESTIONS

10 / 10 completed

Single Sorted Linked List

SINGLE SORTED LINKED LIST

"Two departments maintain separate sorted lists of student IDs. For centralized processing, both lists need to be merged while preserving the sorted order. Write a C program to merge two sorted singly linked lists into a single sorted linked list.

Test Case 1

Input

List 1: 10 → 30 → 50

List 2: 20 → 40 → 60

Output

10 → 20 → 30 → 40 → 50 → 60 → NULL

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 // Insert at end
13 void insertEnd(int value)
14 {
15     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->next = NULL;
18
19     if (head == NULL)
20     {
21         head = newNode;
```

OUTPUT

Total number of nodes = 5

INPUT

10 20 30 40 50

QUESTIONS

10 / 10 completed

Sorted Linked List-2

SORTED LINKED LIST-2

*A database contains sorted customer IDs, but due to data-entry errors, duplicate records have been created. Develop a C program that removes duplicate nodes from a sorted linked list and displays the updated list.

Test Case 1

Input

10 → 10 → 20 → 20 → 30 → 30

Output

10 → 20 → 30 → NULL*

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 struct Node *head = NULL;
11
12 // Insert at end
13 void insertEnd(int value)
14 {
15     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->next = NULL;
18
19     if (head == NULL)
20     {
21         head = newNode;
```

OUTPUT

10 -> 20 -> 30 -> NULL

INPUT

10 10 20 20 30 30

QUESTIONS

10 / 10 completed

Structure

STRUCTURE

Write a C program to store student details (Roll No, Name, Marks) using structure and save them into a file. Display highest mark, class average, and total records stored.

Sample Input:

Enter number of students: 2

101 Ravi 75

102 Kiran 85

Sample Output:

Highest Mark: 85 (Kiran)

Class Average: 80.00

2 records stored successfully.

Test Case:

1 student entry → Average = Marks of that student

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 struct Student {
4     int roll;
5     char name[50];
6     int marks;
7 };
8
9 int main() {
10     struct Student s[100];
11     FILE *fp;
12     int n, i, total = 0, max;
13     float avg;
14
15     scanf("%d", &n);
16
17     fp = fopen("student.txt", "w");
18
19     for (i = 0; i < n; i++) {
20         scanf("%d %s %d", &s[i].roll, s[i].name, &s[i].marks);
21         fprintf(fp, "%d %s %d\n", s[i].roll, s[i].name, s[i].marks);
```

OUTPUT

Security Error: Disallowed code pattern detected.

INPUT

2

QUESTIONS

10 / 10 completed

Fib series-iterative & recur

FIB SERIES-ITERATIVE & RECUR

Write a C program to generate Fibonacci series up to N terms using both iterative and recursive methods. The program should display both outputs and compare execution time in milliseconds. Also display total recursive calls made.

Sample Input:

Enter N: 7

Sample Output:

Iterative: 0 1 1 2 3 5 8

Recursive: 0 1 1 2 3 5 8

Recursive Calls: 13

Iterative Time: 0.001 ms

Recursive Time: 0.005 ms

Test Case:

Enter N: 1

Output:

0

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, arr[100], i, j, temp;
5
6     scanf("%d", &n);
7
8     for (i = 0; i < n; i++)
9         scanf("%d", &arr[i]);
10
11     // Sort the array
12     for (i = 0; i < n - 1; i++) {
13         for (j = i + 1; j < n; j++) {
14             if (arr[i] > arr[j]) {
15                 temp = arr[i];
16                 arr[i] = arr[j];
17                 arr[j] = temp;
18             }
19         }
20     }
21 }
```

OUTPUT

2 5 6 8

INPUT

QUESTIONS

10 / 10 completed

Polynomials

POLYNOMIALS

"In scientific computing applications, mathematical polynomials are often represented using linked lists where each node stores a coefficient and exponent. Write a C program to represent two polynomials using linked lists and perform polynomial addition.

Test Case 1

Input

Polynomial 1: $5x^2 + 3x + 2$ Polynomial 2: $4x^2 + 6x + 1$

Output

Resultant Polynomial:

 $9x^2 + 9x + 3$

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int coeff;
6     int exp;
7     struct Node *next;
8 };
9
10 struct Node* createNode(int c, int e) {
11     struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
12     newNode->coeff = c;
13     newNode->exp = e;
14     newNode->next = NULL;
15     return newNode;
16 }
17
18 void insert(struct Node **head, int c, int e) {
19     struct Node *newNode = createNode(c, e);
20     if (*head == NULL) {
21         *head = newNode;
```

OUTPUT

```
Resultant Polynomial:
9x^2 + 9x^1 + 3x^0
```

INPUT

QUESTIONS

10 / 10 completed

Doubly Linked List

DOUBLY LINKED LIST

"A music player maintains a playlist where users can move both forward and backward between songs. This can be efficiently implemented using a doubly linked list. Write a C program to create a doubly linked list and display nodes in forward and backward directions.

Test Case 1

Input

10 → 20 → 30 → 40

Output

Forward Traversal : 10 20 30 40
Backward Traversal: 40 30 20 10
"

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *prev, *next;
7 };
8
9 int main() {
10     int n, i, val;
11     struct Node *head = NULL, *temp, *newNode, *last = NULL;
12
13     scanf("%d", &n);
14
15     for(i = 0; i < n; i++) {
16         scanf("%d", &val);
17
18         newNode = (struct Node*)malloc(sizeof(struct Node));
19         newNode->data = val;
20         newNode->prev = NULL;
21         newNode->next = NULL;
```

OUTPUT

```
Forward Traversal : 20 30 40 40 40 40 40 40 40 40
Backward Traversal: 40 40 40 40 40 40 40 40 30 20
```

INPUT